Práctica de aprendizaje automático (parte 1)

Autores:

- Alejandro Martínez
- Paula Pérez
- Antón Salvadores

Asignatura: Seminario en Métodos Cuantitativos
Máster en Banca y Finanzas Cuantitativas
Curso: 2025/26

1. Construcción de clasificadores en bases de datos sintéticas

Entrega este cuaderno habiendo **ejecutado todas las celdas**. Incluye en la entrega **todos los ficheros necesarios para su ejecución**.

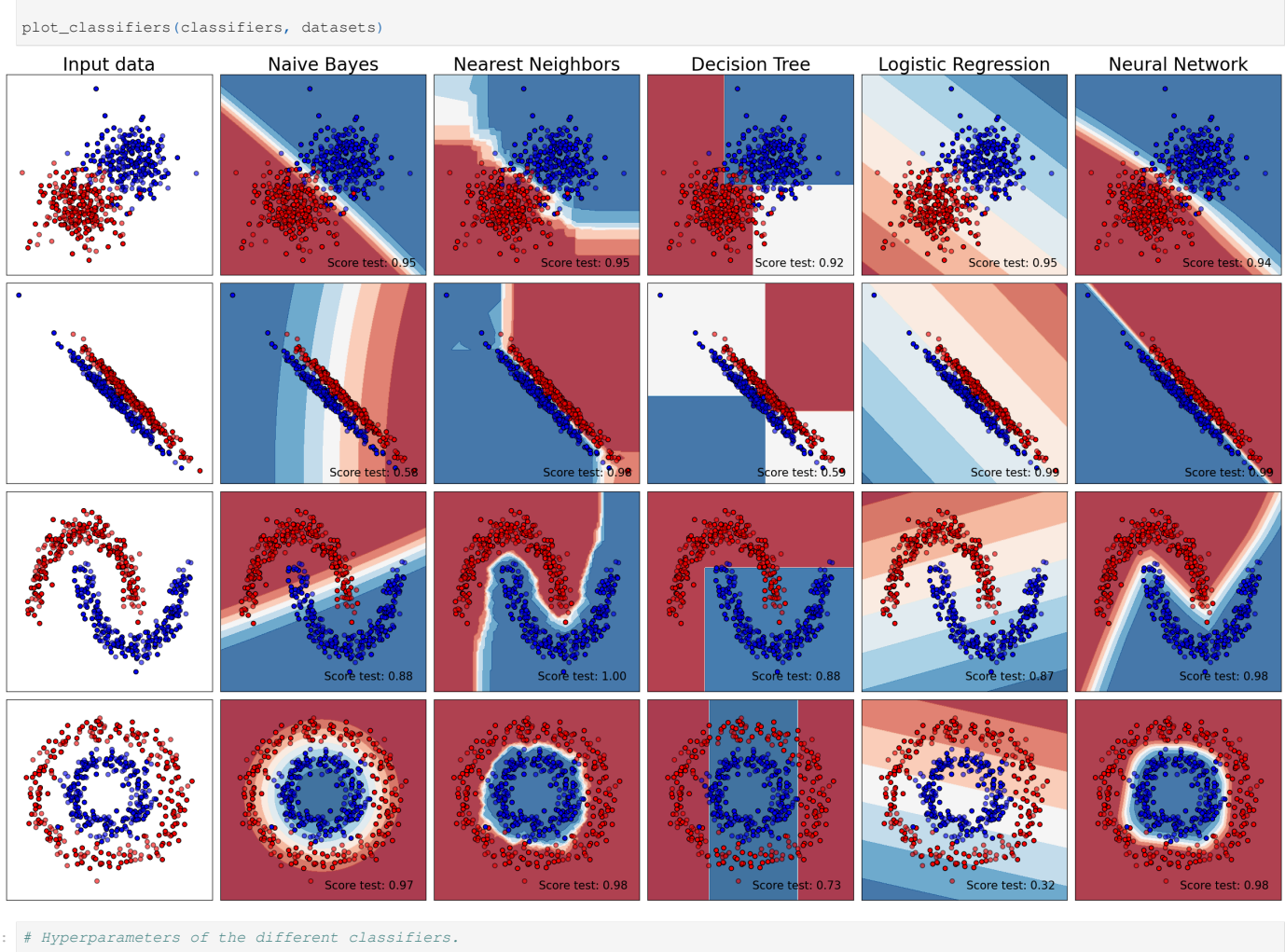
Asegúrate de que la presentación está bien estructurada: Se valorará la **claridad, concisión, y completitud** del informe.

```
# Fuente:
# http://scikit-learn.org/stable/auto_examples/classification/plot_classifier_comparison.html
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs, make_moons, make_circles
from sklearn.model_selection import cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import RobustScaler
from sklearn.pipeline import Pipeline

import sys
sys.path.insert(0, './tools')

# Subir archivo
from tools_assignment_1 import plot_classifiers, plot_error, plot_tree_depths

%matplotlib inline
```

```
# Datos sintéticos

simple = make_blobs(n_samples=500, n_features=2,
                    centers=[[0,0], [2.5,2.5]],
                    random_state=1)
X,y = make_blobs(n_samples=500, n_features=2,
                 centers=[[0,0], [5,1]],
                 random_state=1)
X = X @ np.asarray([[1,-2],[-20,10]])
linearly_separable=(X,y)

datasets = [
    simple,
    linearly_separable,
    make_moons(noise=0.1, random_state=0, n_samples=500),
    make_circles(
        noise=0.1,
        factor=0.5,
        random_state=1,
        n_samples=500
    )
]
```

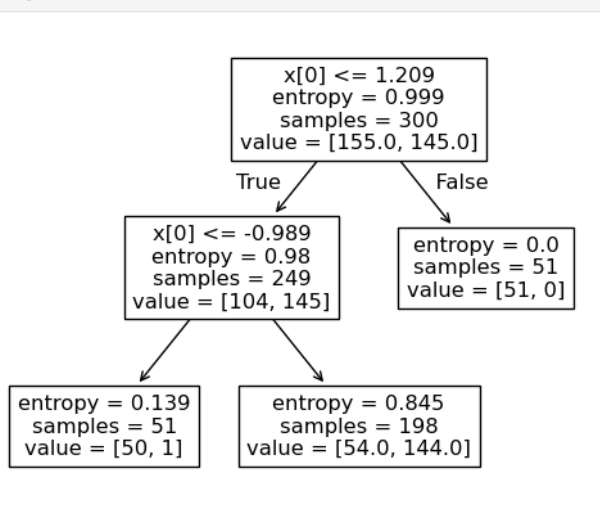
```
classifiers = [("Naive Bayes", GaussianNB()),
               ("Nearest Neighbors",
                KNeighborsClassifier(p=1, metric='minkowski', n_neighbors=7, weights='distance')),
               ("Decision Tree",
                DecisionTreeClassifier(criterion='entropy',
                                       max_depth=2)),
               ("Logistic Regression",
                LogisticRegression(C=1.0e10, solver='lbfgs')),
               # C: controls the level of regularization (larger C, less regularized)
               ("Neural Network",
                MLPClassifier(hidden_layer_sizes=(50,),
                              max_iter=1000,
                              alpha=0.0))]

# MLPClassifier: alpha es la constante de regularización, hidden_layer_sizes es el número
# de neuronas por capa, solver no lo tocamos, learning_rate la constante de aprendizaje,

plot_classifiers(classifiers, datasets)
```



```
# Hyperparameters of the different classifiers.
for name, clf in classifiers:
    print(name, "->", clf)
```

```
Naive Bayes -> GaussianNB()
Nearest Neighbors -> KNeighborsClassifier(n_neighbors=7, p=1, weights='distance')
Decision Tree -> DecisionTreeClassifier(criterion='entropy', max_depth=2)
Logistic Regression -> LogisticRegression(C=10000000000.0)
Neural Network -> MLPClassifier(alpha=0.0, hidden_layer_sizes=(50,), max_iter=1000)
```

```
# Decision tree: Structure, hyperparameters, and node properties.
_ = plot_tree(classifiers[2][1])
```



Ejercicio 1.1: Exploración de vecinos próximos.

1. Haz una gráfica de la dependencia del error de clasificación en función del número de vecinos próximos que se utilizan para realizar la predicción, con `n_neighbors = (1, 3, 7, 11, 15, 21, 31, 51, 101, ...)`.

2. Número de vecinos en k-nn.

   - ¿por qué se explora el hiperparámetro `n_neighbors` en una escala aproximadamente logarítmica?
   - ¿Debería ser impar cuando hay dos clases? Justifica la respuesta.
   - ¿Debería ser impar cuando hay más de dos clases? Justifica la respuesta

3. Comenta la dependencia del error de clasificación con el número de vecinos.

4. Haz una gráfica similar a las obtenidas en este cuaderno con `plot_classifiers(classifiers, datasets)` para cinco de los clasificadores de vecinos próximos entrenados. Elige los cinco que ilustren tus conclusiones.

5. Comenta la forma de la frontera de decisión dependiendo del número de vecinos.

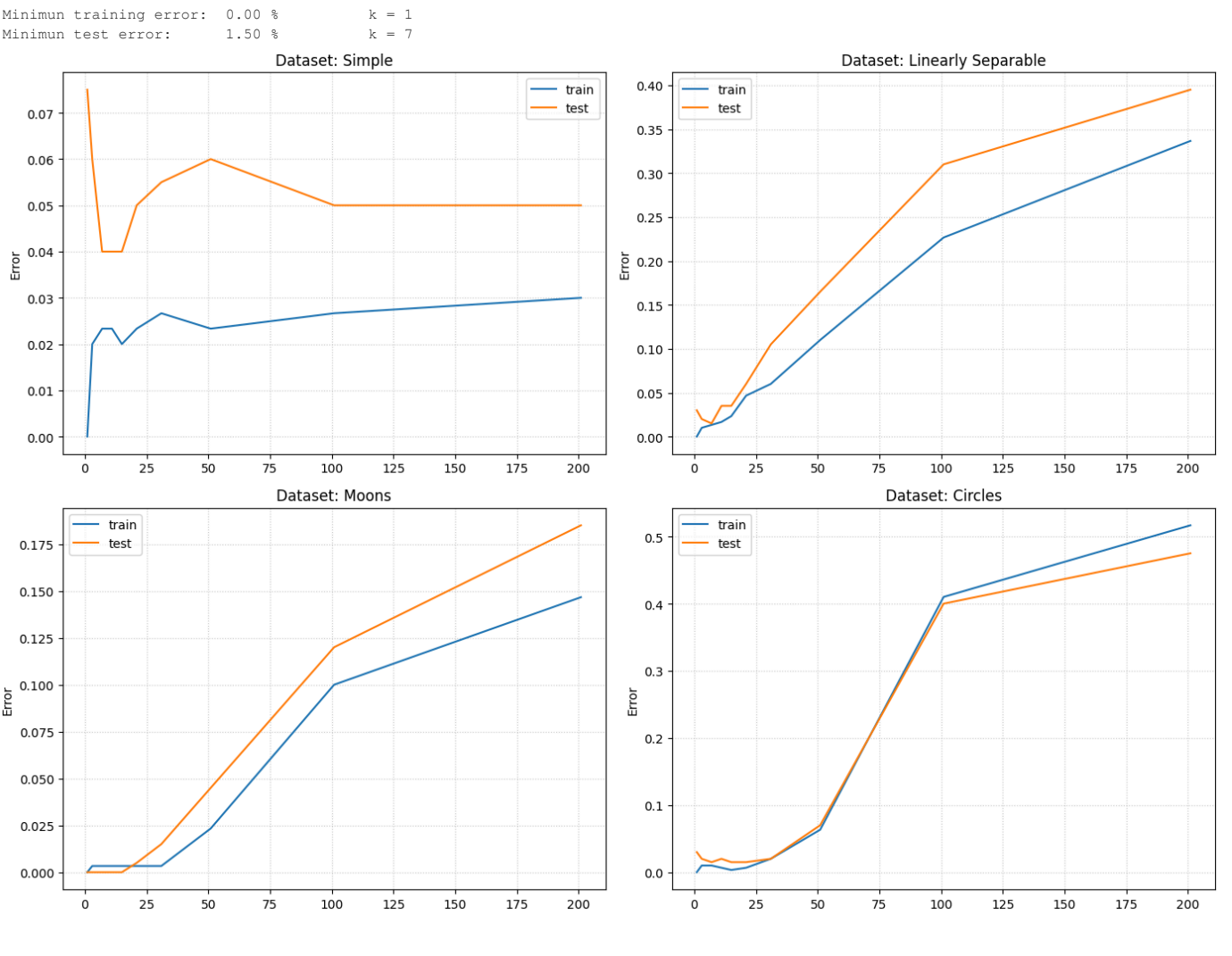
Referencia: https://scikit-learn.org/stable/modules/neighbors.html

Resolución del Ejercicio 1.1

1. **Haz una gráfica de la dependencia del error de clasificación en función del número de vecinos próximos que se utilizan para realizar la predicción, con `n_neighbors = (1, 3, 7, 11, 15, 21, 31, 51, 101, ...)`.**

```
# 1 Gráfica de dependencia del error de clasificación en función del número de vecinos:

# Exploramos el número de vecinos en escala logarítmica en lugar de lineal (de 1 en 1), por ejemplo:
n_neighbors = (1, 3, 7, 11, 15, 21, 31, 51, 101, 201)

nombres = ["Simple", "Linearly Separable", "Moons", "Circles"]

clf_neighbors = [
    Pipeline([
        ('scaler', StandardScaler()),
        ('knn', KNeighborsClassifier(n_neighbors=k, weights='uniform'))
    ])
    for k in n_neighbors
]

plot_error(
    classifiers=clf_neighbors,
    datasets=datasets,
    hyperparameters=n_neighbors,
    names=nombres
)
```

```
--- Dataset: Simple ---
Minimun training error:  0.00 %          k = 1
Minimun test error:      4.00 %          k = 7
--- Dataset: Linearly Separable ---
Minimun training error:  0.00 %          k = 1
Minimun test error:      1.50 %          k = 7
--- Dataset: Moons ---
Minimun training error:  0.00 %          k = 1
Minimun test error:      0.00 %          k = 1
--- Dataset: Circles ---
Minimun training error:  0.00 %          k = 1
Minimun test error:      1.50 %          k = 7
```



2. **Número de vecinos en k-nn.**
   - **¿por qué se explora el hiperparámetro `n_neighbors` en una escala aproximadamente logarítmica?**
   - **¿Debería ser impar cuando hay dos clases? Justifica la respuesta.**
   - **¿Debería ser impar cuando hay más de dos clases? Justifica la respuesta**

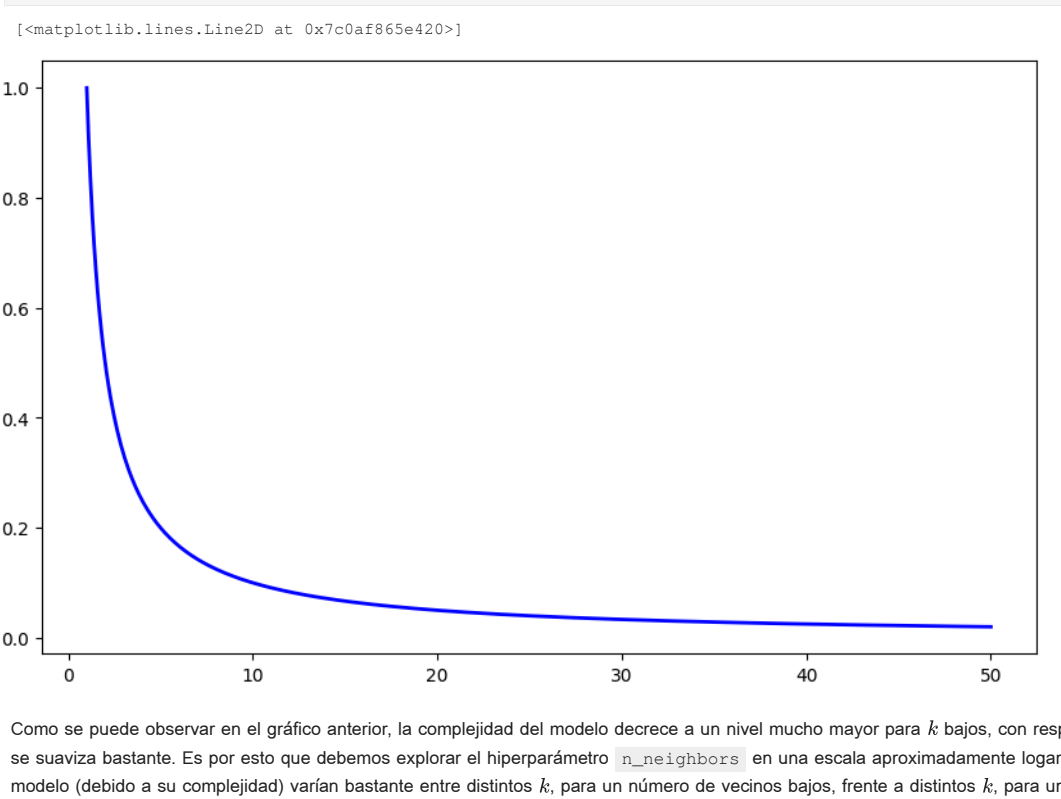
- **¿Por qué se explora el hiperparámetro `n_neighbors` en una escala aproximadamente logarítmica?**

El hiperparámetro `n_neighbors` se explora en una escala aproximadamente logarítmica, ya que la sensibilidad del modelo al número de vecinos baja drásticamente, a medida que aumentamos k. La complejidad del modelo en vecinos próximos la podemos determinar como $\frac{1}{k}$, siendo k el número de vecinos.

Evidentemente, si $k = 1 \Rightarrow \frac{1}{k} = 1$, la complejidad del modelo es máxima, si $k \to \infty \Rightarrow \frac{1}{k} = 0$, la complejidad del modelo es mínima.

```
k = np.linspace(1, 50, 500)
plt.figure(figsize=(10, 6))
plt.plot(k, 1 / k, color='blue', linewidth=2, label=r'$f(k) = \frac{1}{k}$')
```

```
Out[ ]: [<matplotlib.lines.Line2D at 0x7c0af865e420>]
```



Como se puede observar en el gráfico anterior, la complejidad del modelo decrece a un nivel mucho mayor para k bajos, con respecto a k altos, donde la función se suaviza bastante. Es por esto que debemos explorar el hiperparámetro `n_neighbors` en una escala aproximadamente logarítmica, ya que los resultados del modelo (debido a su complejidad) varían bastante entre distintos k, para un número de vecinos bajos, frente a distintos k, para un número de vecinos alto.

Como ejemplo, para el algoritmo no es lo mismo diferenciar a qué clase pertenece cada región diferenciando entre 1 o 3 vecinos, que diferenciando entre 51 y 53 vecinos. Diferenciar entre 51 y 53 vecinos es prácticamente idéntico (cubrir una región muy amplia), sin embargo, diferenciar entre 1 o 3 vecinos tiene una diferencia mucho más significativa.

- **¿Debería ser impar cuando hay dos clases? Justifica la respuesta.**

En el caso en el que haya solo dos clases ($y \in \{C_1, C_2\}$), como en este caso, si definimos un número de vecinos par, podría haber un empate técnico (un vecino para cada clase) y el algoritmo debería decidir arbitrariamente a qué clase pertenece esa región. Con un número impar de vecinos, siempre habrá una mayoría simple para alguna región, facilitando el trabajo al algoritmo.

- **¿Debería ser impar cuando hay más de dos clases? Justifica la respuesta**

En caso de haber m clases ($y \in \{C_1, \ldots, C_m\}$) con $m > 2$, como por ejemplo tres, podría haber un empate técnico aún siendo impar el hiperparámetro k. Continuando con el ejemplo y tres vecinos, cada vecino podría pertenecer a una clase y el algoritmo debería decidir arbitrariamente a qué clase pertenece la región. En este caso, la imparidad en el número de vecinos no asegura la imposibilidad de empate entre clases.

Sin embargo, deberíamos seguir empleando un número impar de vecinos, ya que hay un gran número de casos, sobretodo en las fronteras de decisión, donde el problema se decide igualmente entre dos clases. En un problema con tres clases, A,B y C por ejemplo, la frontera de decisión involucrará probablemente a dos de las tres clases en la mayoría de los casos, difícilmente a las tres simultáneamente. Al convertirse localmente en un problema binario de nuevo, la imparidad nos ayuda a resolver estos empates, como en el ejemplo anterior. Una sugerencia para desempatar podría ser usar el hiperparámetro *weights='distance'*. Esto hace que el vecino más cercano tenga un peso mayor en el recuento de votos, esto hace que sea prácticamente imposible que haya un empate ya que ambos vecinos tendrían que estar exactamente a la misma distancia de un punto específico.

Por tanto, por norma general, siempre emplearemos un número de vecinos impar, para problemas de cualquier clase.

3. Comenta la dependencia del error de clasificación con el número de vecinos.

Antes de hacer ninguna observación, primero observemos los datos y la manera en la que están generados. Esto nos dará ciertas pistas para comentar la dependencia del error de clasificación con el número de vecinos, así como para los demás modelos.

En concreto, tenemos 4 tipos de datos; simples, linealmente separables, en forma de media luna y en círculos, con solo dos clases diferenciadas. Si observamos los gráficos, ambas clases en todos los gráficos están claramente diferenciadas, salvo en el conjunto de datos simples, donde sí que hay algunos puntos que se entrelazan con la región que no es de su clase.

Si observamos cómo están generados los datos, esto tiene una razón detrás. Cada clase está generada alrededor de un centro muy distante, y con muy poca variabilidad alrededor de este. Esto lo podemos observar en la generación del conjunto de datos simple (`centers=[[0,0], [2.5,2.5]]`) y en el conjunto de datos linealmente separables (`centers=[[0,0], [5,1]]`). En cuanto a la generación de las medias lunas y de los círculos, esto se observa en el argumento `noise=0.1` , que significa que los datos generados de cada clase tienen una desviación típica de 0.1, la cual es una variabilidad muy baja entre miembros de cada clase. Para los círculos, a mayores, se tiene un factor de escala, para la distancia entre el círculo interior y exterior (`factor=0.5`), que parece un factor suficientemente grande como para diferenciar ambos círculos correctamente, a juicio de los gráficos.

Esto viene a decir que ambas clases están bastante bien diferenciadas, además de ser bastante homogéneas entre ellas, como se puede observar. Por tanto, al hacer la separación en los datos entre entrenamiento y test, realmente son muy parecidos entre sí, por lo que quizás nuestras predicciones, si bien parece que funcionan correctamente (tenemos errores muy bajos para todos los datos en vecinos próximos, como se pueden observar en los gráficos del enunciado), no son lo suficientemente precisas.

Esta exploración en la generación de los datos, concuerda con lo que observamos en la evolución de los errores de clasificación para todos los conjuntos. De nuevo, en el conjunto de datos simples, la evolución del error de entrenamiento y de test, es la más "lógica" entre todos los datos, pues como habíamos comentado previamente, es el conjunto de datos más heterogéneo (sin serlo demasiado) de entre los cuatro considerados.

En *KNNeighbors* una forma de medir la complejidad podría ser el cociente $1/k$, de esta forma, cuanto menor sea k, el modelo es más complejo, i.e, más se ajusta a las fluctuaciones del conjunto de datos de entrenamiento y, por tanto, más difícil le resulta identificar patrones fuera de esos datos en particular (el rol que juegan los datos de test). A mayor k, el modelo es más simple, menos se ajusta a los datos de entrenamiento en particular y puede capturar tendencias más trascendentes en los datos.

Esto es lo que se conoce como Bias-variance trade off: los modelos más complejos tienen menor sesgo (pues se ajustan muy bien a los datos de entrenamiento) pero mayor varianza (ya que no son capaces de capturar fluctuaciones de estos datos), se ajustan al ruido (Overfitting), logrando error 0 en train pero fallando en el test. Por otro lado, los modelos más simples tienen mayor sesgo (diferencian peor para el modelo de entrenamiento) pero tienen menor varianza, en estos casos la predicción del modelo tiende a la media global (Underfitting), ignorando la estructura local de los datos. Estas son las dos componentes que forman parte del error de predicción. El objetivo sería encontrar un punto medio entre estos, sin tener ni demasiada varianza ni demasiado sesgo.

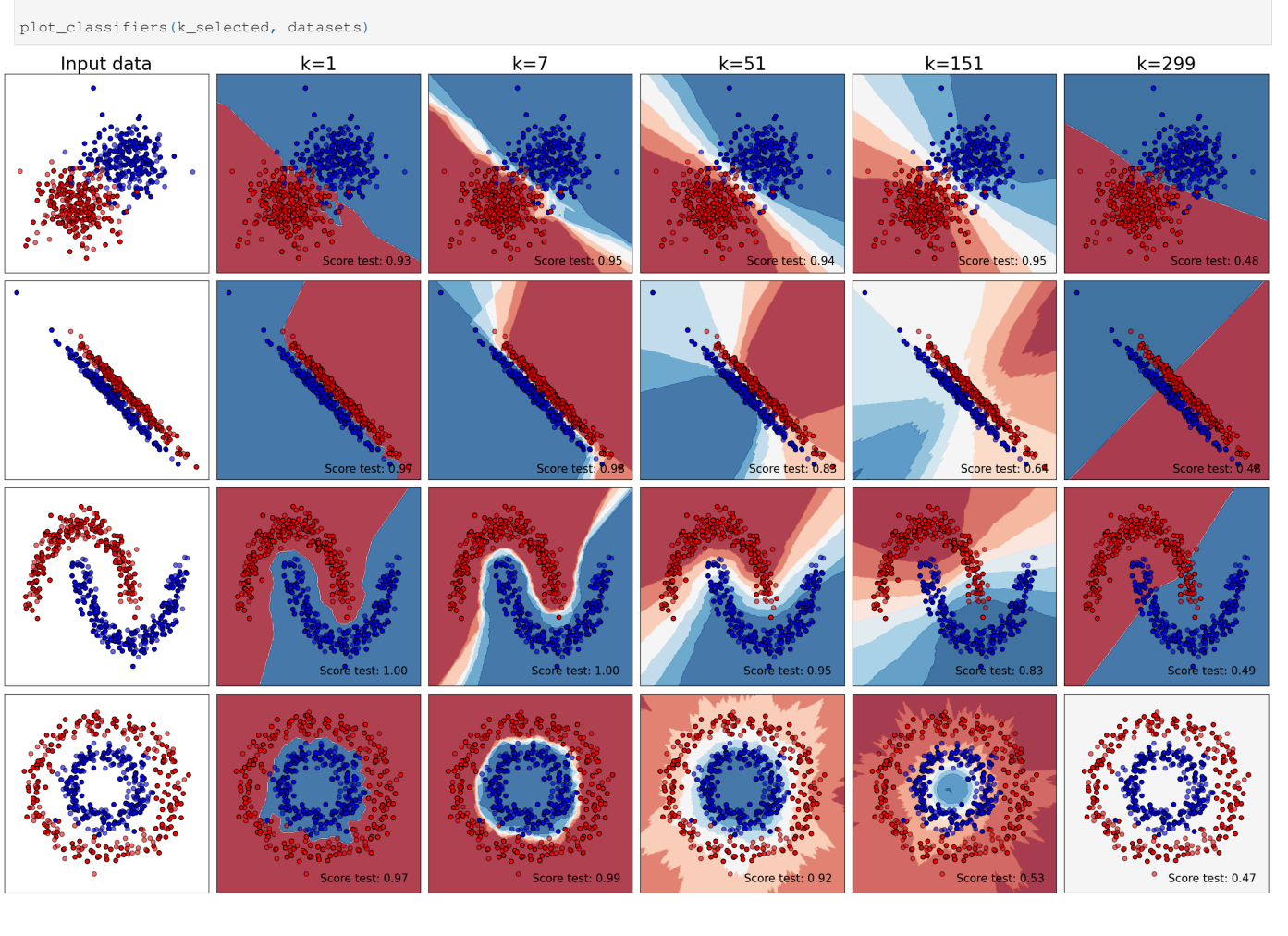
Así, el gráfico del error para los datos de entrenamiento suele ser una tendencia monótona creciente (para $k = 1$, el modelo más complejo, prácticamente no tiene errores, a medida que el modelo se simplifica, comete más) y el gráfico del error para los datos de test suele tener forma de J o de U, donde el punto mínimo es en donde se encuentra el equilibrio entre ambos tipos de errores, como hemos explicado.

En nuestro caso, el error de entrenamiento muestra una tendencia estrictamente creciente. En test, la curva dibuja una 'U' muy asimétrica: desciende ligeramente hasta el óptimo para luego crecer. Que el mínimo en test ($k = 7$) esté tan cerca de la máxima complejidad ($1/k = 1$) confirma la baja variabilidad y limpieza de los datos generados. Al existir una separación tan clara entre clases y tan poco ruido, el modelo tolera una alta complejidad sin sufrir overfitting, convirtiendo al modelo de entrenamiento en un buen predictor del test. Esto se traduce en un mínimo de error en entrenamiento con $k = 1$ y en test con $k = 7$ (para la mayoría de los conjuntos), alcanzando en todos los casos precisiones cercanas al 100%.

Nota: en el gráfico de `plot_classifiers` del enunciado, se utiliza la distancia de Manhattan para ponderar la distancia entre los vecinos próximos, nosotros estamos utilizando una ponderación uniforme, en todos los apartados. Mención por posibles diferencias en los resultados con el enunciado.

4. **Haz una gráfica similar a las obtenidas en este cuaderno con `plot_classifiers(classifiers, datasets)` para cinco de los clasificadores de vecinos próximos entrenados. Elige los cinco que ilustren tus conclusiones.**

```
# 4. Gráfica para cinco clasificadores de vecinos próximos
# con distinto número de vecinos.

# Seleccionamos clasificadores
k_selected = [
    ("k=1",   KNeighborsClassifier(n_neighbors=1)),
    ("k=7",   KNeighborsClassifier(n_neighbors=7)),
    ("k=51",     KNeighborsClassifier(n_neighbors=51)),
    ("k=151",   KNeighborsClassifier(n_neighbors=151)),
    ("k=299",   KNeighborsClassifier(n_neighbors=299))
]

plot_classifiers(k_selected, datasets)
```



5. **Comenta la forma de la frontera de decisión dependiendo del número de vecinos.**

A raíz de las conclusiones del ejercicio anterior, se ha considerado el dataset circular y los siguientes 5 valores para el hiperparámetro k:

- $k = 1$: El menor valor de k corresponde con el modelo más complejo posible ya que este solo necesita saber las características del vecino más próximo para colorear la frontera. Pese a que en este ejemplo obtiene una tasa de acierto (*score test*) muy alta, podemos observar cómo la frontera de decisión es la más "nerviosa" de las 5, esto es que el modelo está penalizando la alta varianza que soporta el modelo con $k = 1$ en contraposición con el bajo sesgo (mucha complejidad en el trazado de las fronteras).

- $k = 7$: En $k = 7$ se encuentra el óptimo del trade-off entre sesgo y varianza. En este valor de k vimos en el apartado anterior cómo **el error en test era mínimo**, alcanzando un (*score test*) máximo. En este punto el modelo es lo suficientemente sofisticado como para captar la distribución circular de los datos y lo suficientemente robusto como para eliminar los efectos del ruido y la extrema irregularidad de las fronteras de decisión causadas por el sobreajuste. En otras palabras, el modelo encuentra en 7, la "mayoría óptima" donde el sesgo y la varianza toman los valores más bajos posibles sin descompensarse el uno al otro, y por tanto el punto donde el modelo mejor generaliza.

- $k = 51$: En este nivel, nos encontramos en una zona de baja complejidad. El modelo empieza a tener un sesgo alto, ya que la frontera es tan suave que empieza a ignorar la geometría real de los datos (el círculo) para volverse una forma mucho más simple. El clasificador ya no es capaz de capturar las dependencias locales y la "limpieza" de la frontera se traduce en una pérdida de precisión en el test. La tonalidad de los colores representa la "confianza" del modelo en colorear un pixel en específico. Podemos ver ya como en este valor de k, los tonos rojizos comienzan a ser mucho más suaves porque al preguntar a 51 vecinos para decidir el color de un píxel se "cuelan" muchos puntos azules en la votación y el modelo se confunde.

- $k = 151$: La complejidad del modelo es extremadamente pequeña. Podemos ver como para este valor de k, el modelo ha disparado enormemente su error en el test. Esto se debe a que el modelo es demasiado "simple" para entender cualquier patrón. En este estado, el modelo ignora completamente la información de los atributos x y simplemente colorea todo el espacio con el color de la clase que más abunda, esto se traduce en un sesgo alto y una varianza baja, de hecho en este punto se puede ver como el modelo acierta prácticamente todos los puntos rojos y falla prácticamente todos los puntos azules, el modelo no ha logrado captar la forma geométrica circular de los datos y por eso sale un (*score test*) aproximadamente del 50%

- $k = 299$. Este es el valor límite respecto al tamaño del entrenamiento ($N = 300$), nos encontramos en el punto de menos complejidad posible, donde el clasificador pierde la localidad y evalúa cada punto mediante una votación global sobre todo el dataset. Dado el balanceo de clases, esto deriva en un empate técnico constante. El color blanco del fondo en el caso del dataset circular denota esta máxima incertidumbre (baja confianza), donde el modelo estima una probabilidad $P(y|x) \approx 0.5$ en todo el espacio, limitándose a replicar la proporción *a priori* de las clases sin capacidad real de discriminación.

Ejercicio 1.2: Exploración de árboles de decisión.

1. Haz una gráfica de la dependencia del error de clasificación en función de la profundidad máxima del árbol de decisión `max_depth = (1, 2, 3, 4, 5,...)`.

2. Profundidad máxima

   - ¿por qué se explora el hiperparámetro `max_depth` es una escala lineal?
   - Haz una gráfica de la profundidad del árbol generado en función de `max_depth`.

3. Comenta la dependencia del error de clasificación con la profundidad máxima del árbol.

4. Haz una gráfica similar a las obtenidas en este cuaderno con `plot_classifiers(classifiers, datasets)` para cinco de los distintos árboles entrenados. Elige los cinco que ilustren tus conclusiones.

5. Comenta la forma de la frontera de decisión dependiendo de la profundidad máxima del árbol.

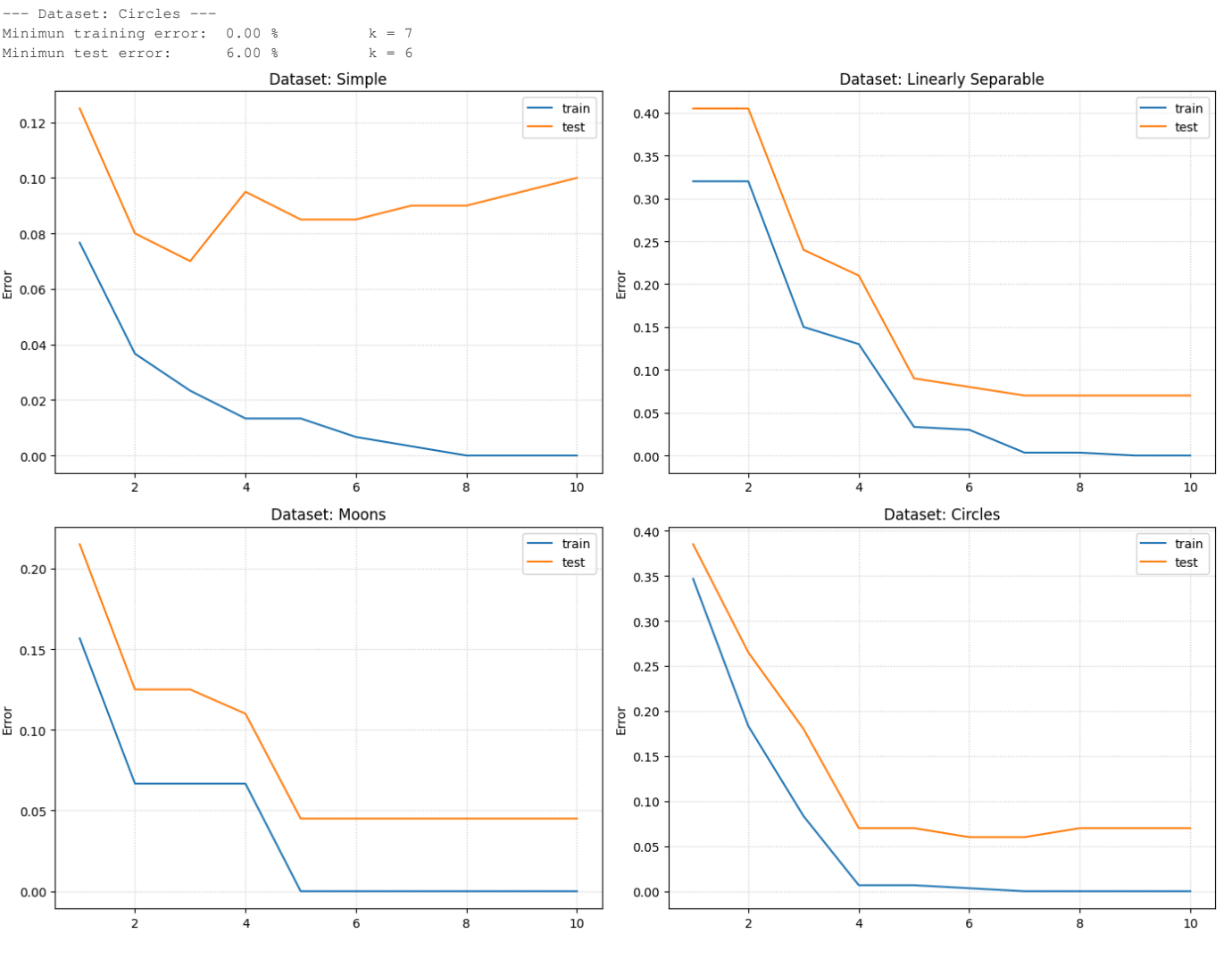
Referencia: https://scikit-learn.org/stable/modules/tree.html#classification

Resolución del Ejercicio 1.2

1. **Haz una gráfica de la dependencia del error de clasificación en función de la profundidad máxima del árbol de decisión `max_depth = (1, 2, 3, 4, 5,...)`.**

```
# 1 Gráfica de dependencia del error de clasificación en función
# de la profundidad máxima del árbol de decisión.

max_depth = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
clf_tree = [
    DecisionTreeClassifier(criterion='entropy', max_depth=d)
    for d in max_depth
]

plot_error(
    classifiers=clf_tree,
    datasets=datasets,
    hyperparameters=max_depth,
    names=nombres
)
```

```
--- Dataset: Simple ---
Minimun training error:  0.00 %          k = 8
Minimun test error:      7.00 %          k = 3
--- Dataset: Linearly Separable ---
Minimun training error:  0.00 %          k = 9
Minimun test error:      7.00 %          k = 7
--- Dataset: Moons ---
Minimun training error:  0.00 %          k = 5
Minimun test error:      4.50 %          k = 5
--- Dataset: Circles ---
Minimun training error:  0.00 %          k = 7
Minimun test error:      6.00 %          k = 6
```



2. **Profundidad máxima**
   - **¿por qué se explora el hiperparámetro `max_depth` es una escala lineal?**
   - **Haz una gráfica de la profundidad del árbol generado en función de `max_depth`.**

- **¿Por qué se explora el hiperparámetro `max_depth` es una escala lineal?**

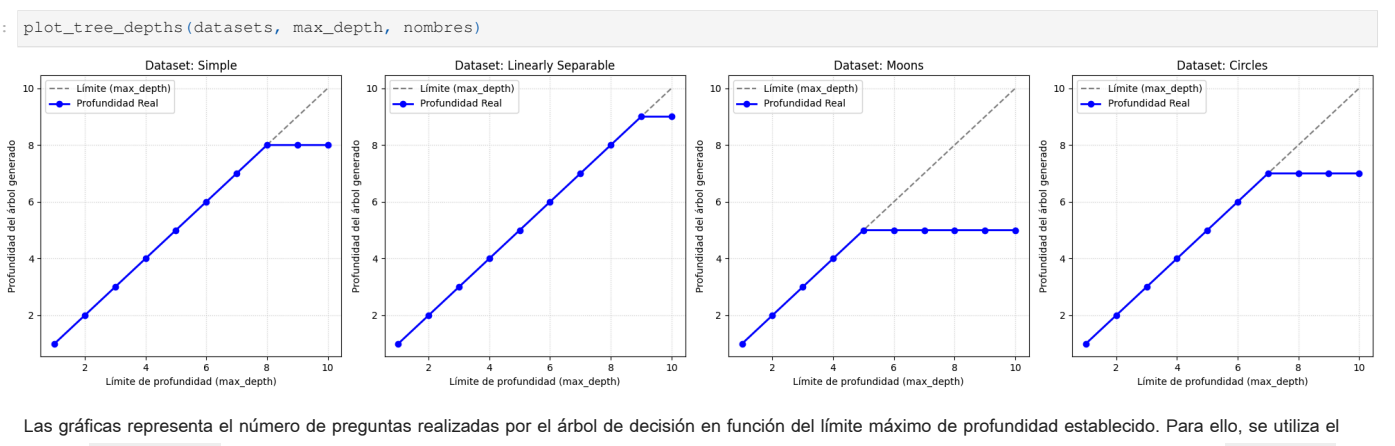
El hiperparámetro `max_depth` de la clase DecisionTreeClassifier codifica la profundidad máxima del árbol de decisión. En otras palabras, mide el número de niveles que hay desde el nodo raíz (la primera pregunta) hasta la hoja (última pregunta). En cada nivel se plantea una pregunta sobre los datos.

A diferencia de otros hiperparámetros, es preferible usar una escala lineal para `max_depth` y, en concreto, números enteros. El motivo se debe a la imposibilidad de hacer preguntas parciales. Por ejemplo, `max_depth` $= 2.6$, implica hacer dos preguntas completas y 0.6. Por lo tanto, no tiene una interpretación clara en términos de niveles de decisión. El propio algoritmo, redondea este tipo de valores continuos a números enteros.

En consecuencia, explorar este hiperparámetro en escala lineal es coherente, ya que cada incremento unitario añade una pregunta "completa" al proceso de clasificación, modificando la complejidad del modelo.

- **Haz una gráfica de la profundidad del árbol generado en función de `max_depth`.**

```
plot_tree_depths(datasets, max_depth, nombres)
```



Las gráficas representa el número de preguntas realizadas por el árbol de decisión en función del límite máximo de profundidad establecido. Para ello, se utiliza el método `get_depth()` como medida de la profundidad efectiva y que cumple la función de ser un contador. En el eje X se representa el valor de `max_depth` y en el eje Y la profundidad realmente alcanzada (`real_depth`).

En todos los conjuntos de datos se observa un comportamiento común: hasta una profundidad aproximadamente 9 se cumple que `real_depth = max_depth`. Este resultado es coherente, ya que hasta dicho valor el árbol encuentra particiones relevantes o informativas y, por tanto, motivos para seguir realizando preguntas. Sin embargo, a partir de ese punto la recta presenta pendiente nula, lo que indica que el árbol deja de crecer al haber alcanzado nodos hoja.

Aunque se permite al modelo aumentar el valor de max_depth, el árbol decide detener su crecimieneto, dado que realizar más preguntas no apoarta información relevante y únicamente incrementaría la complejidad del modelo, reduciendo su interpretabilidad y robustez.

3. **Comenta la dependencia del error de clasificación con la profundidad máxima del árbol.**

A partir de las gráficas mostradas en el apartado 1 se extrae las siguiente conclusiones:

Para valores pequeños de `max_depth` , tanto el error de entrenamiento como el error de test son elevadas, alcanzando su valor más alto. Este comportamiento es característico de una situación de infraajuste (underfitting), derivada del alto sesgo del modelo y poca varianza. Bajo este contexto, el árbol de decisión es simple, altamente interpretable y robusto, pero incapaz de capturar adecuadamente la estructura de los datos.

A medida que aumenta el valor de `max_depth` , en todos los conjuntos de gráficos se distingue un patrón común en relación con el error de entrenamiento. Este disminuye de forma monótona hasta alcanzar el valor cero, aunque en distintos valores de profundidad según el dataset. Este comportamiento ocurre porque a medida que aumenta el número de complejitud, el árbol dispone de más preguntas y, por ende, de más información, por lo que es capaz de ajustarse mejor a los datos haciendo predicciones más acertadas al realizar más preguntas, logrando disminuir el sesgo. En paralelo, el error de test también muestra una tendencia a bajada especialmente al inicio. No obstante, es cierto que para el caso concreto de dataset simple experimenta una ligera subida, mientras que en el resto de los conjuntos tiende a estabilizarse sin llegar a alcanzar el valor mínimo igual a cero.

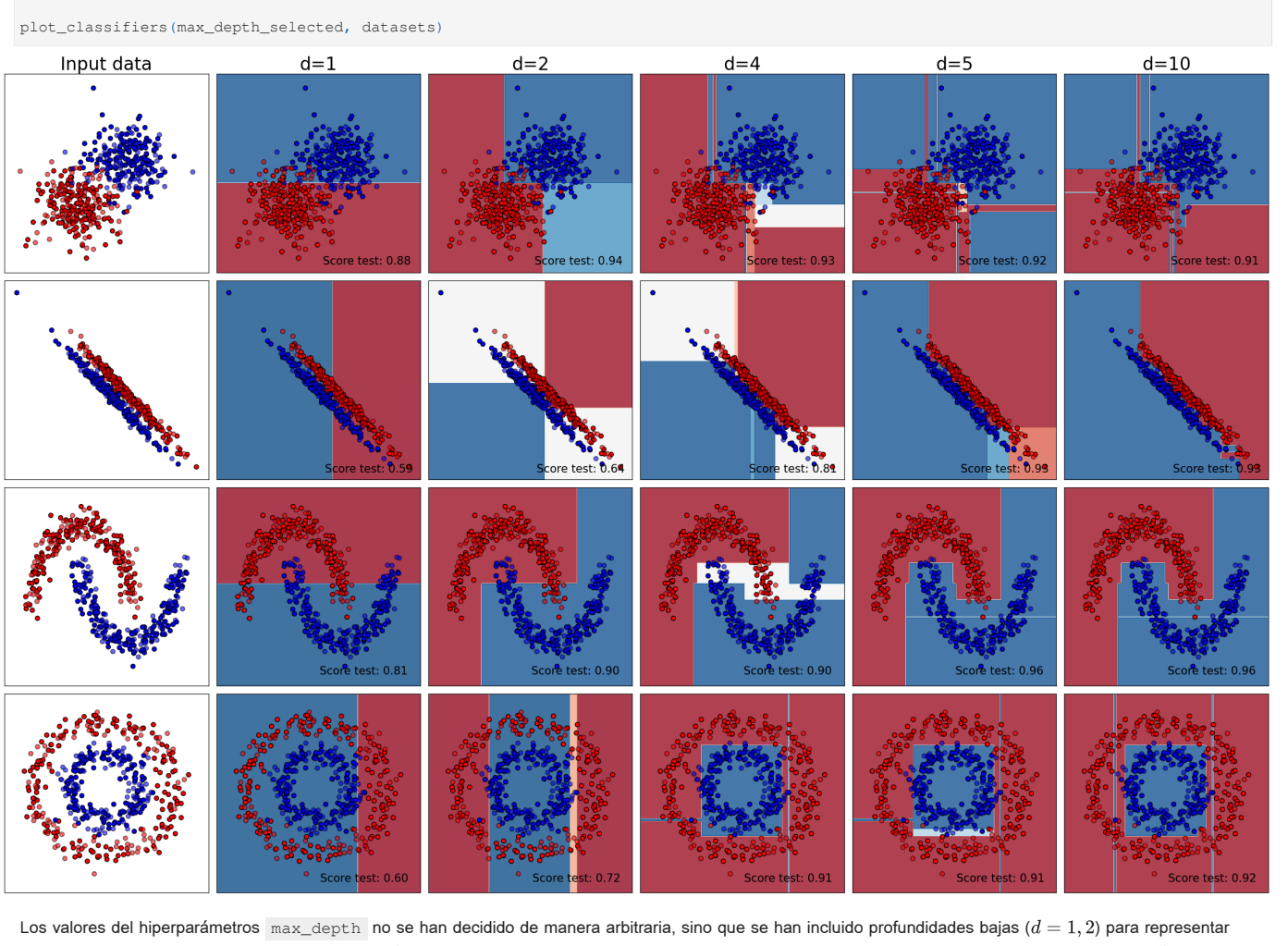
En el extremo contrario, para profundidades grandes, el árbol se convierte en un modelo más complejo donde se produce un sobreajuste (overfitting), que se caracteriza ser un modelo más complejo y tener sesgo bajo pero una alta varianza. Además, poco interpretabilidad y poco robusto. Por ello, aumentar indistintamente el hiperparámetro `max_depth` no es señal de mejora, sino de tener información irrelevante causante que las particiones sean más sensibles a ser menos informativas.

En la práctica, el objetivo es encontrar una complejitud óptima que consiga errores de test bajos, pero al mismo tiempo tenga interpretatibidad. Para ello, la herramienta `max_depth` permite controlar la complejidad del árbol de decisión.

Respecto a las diferencias entre dataset, el comportamiento del error de clasificación en función de `max_depth` también varía según la estructura del dataset. En el dataset simple, el error de test es más sensible a incremento de profundidad, aumentando poco a poco y siendo indicativo de una mayor propensión al sobreajuste. En el conjunto linealmente separable, el error de entrenamiento y el de test evolucionan de forma paralela, y pequeños incrementos en la profundidad producen cambios significativos en ambos errores, reflejando que una complejidad moderada es suficiente para capturar la separación entre clases. Por último, en los dataset moons y circles, al presentar estructuras no lineales, necesitan valores de max_depth superiores para experimentar bajadas apreciable del error de clasificación.

4. **Haz una gráfica similar a las obtenidas en este cuaderno con `plot_classifiers(classifiers, datasets)` para cinco de los distintos árboles entrenados. Elige los cinco que ilustren tus conclusiones.**

```
# 4. Gráfica para cinco clasificadores de vecinos próximos
# con distinto número de vecinos.

max_depth_selected = [
    ("d=1",  DecisionTreeClassifier(max_depth=1)),
    ("d=2",  DecisionTreeClassifier(max_depth=2)),
    ("d=4",  DecisionTreeClassifier(max_depth=4)),
    ("d=5",  DecisionTreeClassifier(max_depth=5)),
    ("d=10",  DecisionTreeClassifier(max_depth=10))
]

plot_classifiers(max_depth_selected, datasets)
```



Los valores del hiperparámetros `max_depth` no se han decidido de manera arbitraria, sino que se han incluido profundidades bajas ($d = 1, 2$) para representar modelos underfitting, profundidades intermedias ($d = 4, 5$) donde se observan cambios más notables en los valores de errores y una profundidad alta ($d = 10$) para visualizar el efecto overfitting.

Estos gráficos nos permiten analizar los efectos de la frontera de decisión cuando pasamos de tener un árbol simple a un árbol complejo.

5. **Comenta la forma de la frontera de decisión dependiendo de la profundidad máxima del árbol.**

Se concluye:

- $d = 1$: Genera el árbol más simple de todos los posibles, realizando tan solo una única pregunta correspondiente al nodo raíz. Bajo este escenario, las fronteras de decisión son poco flexibles e incapaces de ajustarse a la estructura de los datos, especialmente en los conjuntos linearly separable y circles. Como se ha comentado anteriormente, este modelo se carcteriza por estar infrajustado, con sesgo alto y varianza baja, lo que se traduce en tener valores de errores de entrenamiento y test elevados.

- $d = 2$: Al aumentar en una unidad la profundidad, las fronteras de decisión ganan flexibilidad. En particular, los dataset simple y moons comienzan a mostrar un ajuste más adecuado. Sin embargo, la estructura del modelo continua siendo demasiado simple para representar estructuras no lineales más complejos, por lo que el sesgo continúa siendo alto.

- $d = 4, 5$: Ilustran la mejor relación complejidad y capacidad de generación de entre todos los valores de `max_depth` analizados. El árbol dispone de suficiente profundidad para capturar la estructura relevante de los datos, especialmente de los conjuntos moons y circles, generando fronteras ajustadas a la configuración de los datos. En este contexto, el modelo no sobreajustar se encuentra el equilibrio entre sesgo y varianza moderados.

- $d = 10$: Representa el árbol con el mayor valor de profundidad y, por ende, con el mayor número de nodos intermedios. Como consecuencia, el árbol logró capturar con un mayor detalle la estructura de los datos. Por esta razón, los errores de clasificación toman valores bajos, logrando una separación visible entre las distintas clases. Como lado negativo, las fronteras se vuelven más fragmentadas. Además, un mayor número de preguntas puede aportar información irrelevante haciendo que la varianza sea alta.

Ejercicio 1.3: Exploración de redes neuronales

1. Haz una gráfica de la dependencia del error de clasificación de una red neuronal con una sola capa oculta en función del número de neuronas en la capa oculta. `hidden_layer_sizes = ((1,), (2,), (3,),...)`.

2. Comenta la dependencia del error de clasificación con el número de neuronas en la capa oculta.

3. Haz una gráfica similar a las obtenidas en este cuaderno con `plot_classifiers(classifiers, datasets)` para cinco de las distintas redes neuronales entrenadas. Elige las cinco que ilustren tus conclusiones.

4. Comenta la forma de la frontera de decisión dependiendo del número de neuronas en la capa oculta.

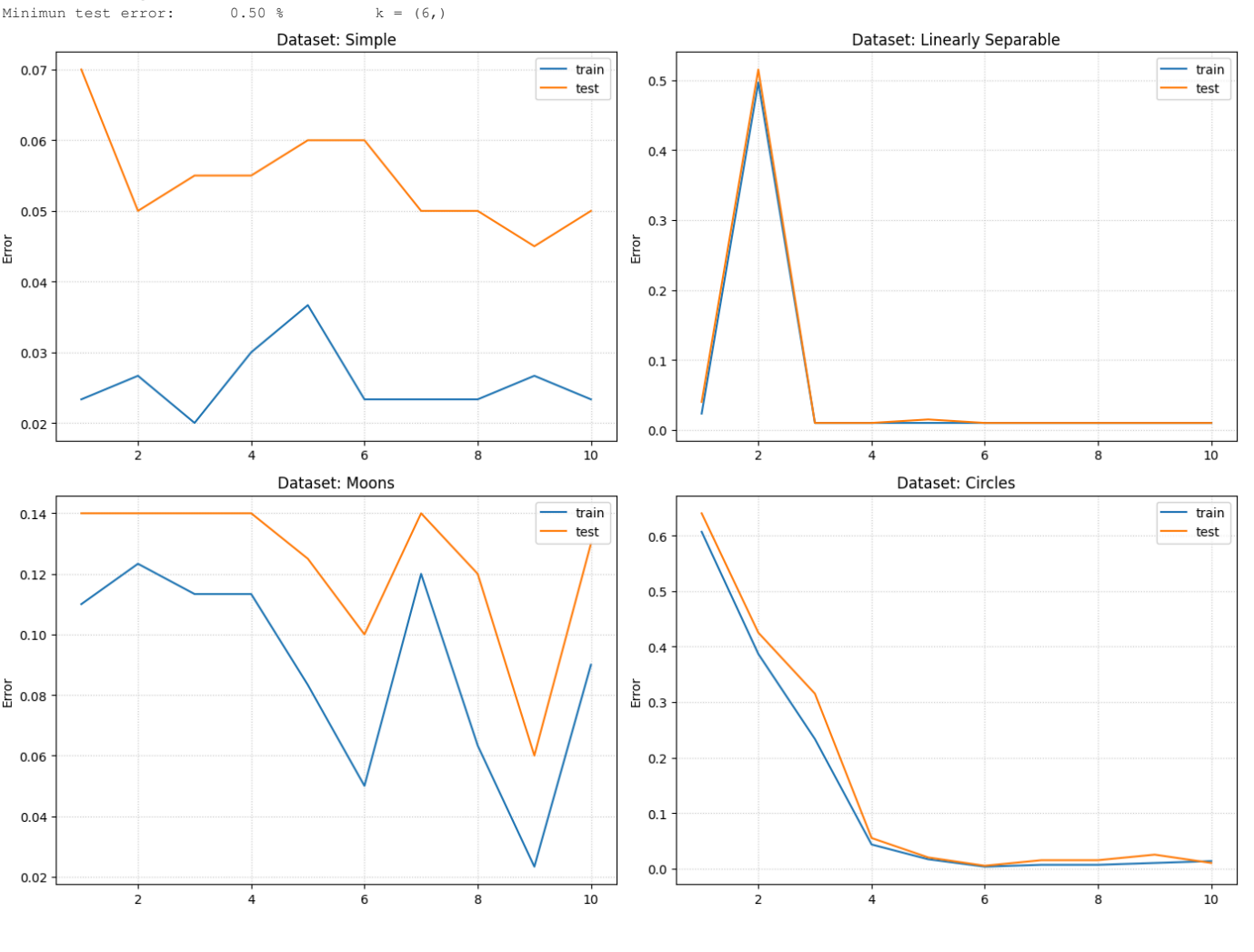
Referencia: https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Resolución del Ejercicio 1.3

1. **Haz una gráfica de la dependencia del error de clasificación de una red neuronal con una sola capa oculta en función del número de neuronas en la capa oculta. `hidden_layer_sizes = ((1,), (2,), (3,),...)`.**

```
# 1 Gráfica de dependencia del error de clasificación en función
# del número de neuronas de la red neuronal.

hidden_layer_sizes = ((1,), (2,), (3,), (4,), (5,), (6,), (7,), (8,), (9,), (10,))
clf_neural_networks = [
    Pipeline([
        ('scaler', StandardScaler()),
        ('mlp', MLPClassifier(
            hidden_layer_sizes=k,
            max_iter=1000,
            alpha=0.0,
            random_state=42
        ))
    ])

    for k in hidden_layer_sizes
]

plot_error(
    classifiers=clf_neural_networks,
    datasets=datasets,
    hyperparameters=hidden_layer_sizes,
    names=nombres
)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
--- Dataset: Simple ---
Minimun training error:  2.00 %          k = (3,)
Minimun test error:      4.50 %          k = (9,)
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
--- Dataset: Linearly Separable ---
Minimun training error:  1.00 %          k = (3,)
Minimun test error:      1.00 %          k = (3,)
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
--- Dataset: Moons ---
Minimun training error:  2.33 %          k = (9,)
Minimun test error:      6.00 %          k = (9,)
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
--- Dataset: Circles ---
Minimun training error:  0.33 %          k = (6,)
Minimun test error:      0.50 %          k = (6,)
```



2. **Comenta la dependencia del error de clasificación con el número de neuronas en la capa oculta.**

En este caso, hemos incluido en la capa oculta desde 1 hasta 10 neuronas. Vemos una tendencia ligeramente distinta a las observadas en los clasificadores anteriores. El error tiende a bajar con el número de neuronas, sobretodo el de train, ya que se ajusta mejor a los datos, aunque de manera irregular, debido a la optimización del modelo. Cabe destacar que el clasificador, cada vez que se ejecute, otorgará unos ajustes diferentes, ya que se inicializa siempre desde dos nodos diferentes, algo a tener en cuenta (por ello se ha fijado la semilla en `random_state=42`).

Para todos los conjuntos de datos, independientemente de cuál sea el mínimo error, observamos una gran caída de este entre 3 y 6 neuronas, por lo que parece que este debe ser el rango cuyo número de neuronas es óptimo, donde el error se minimiza considerando el modelo más simple posible (más robusto). Observamos que, dependiendo del número de neuronas, el error aumenta con el aumento de este, probablemente sea porque la red trata de clasificar correctamente outliers, empeorando la calidad general del modelo.

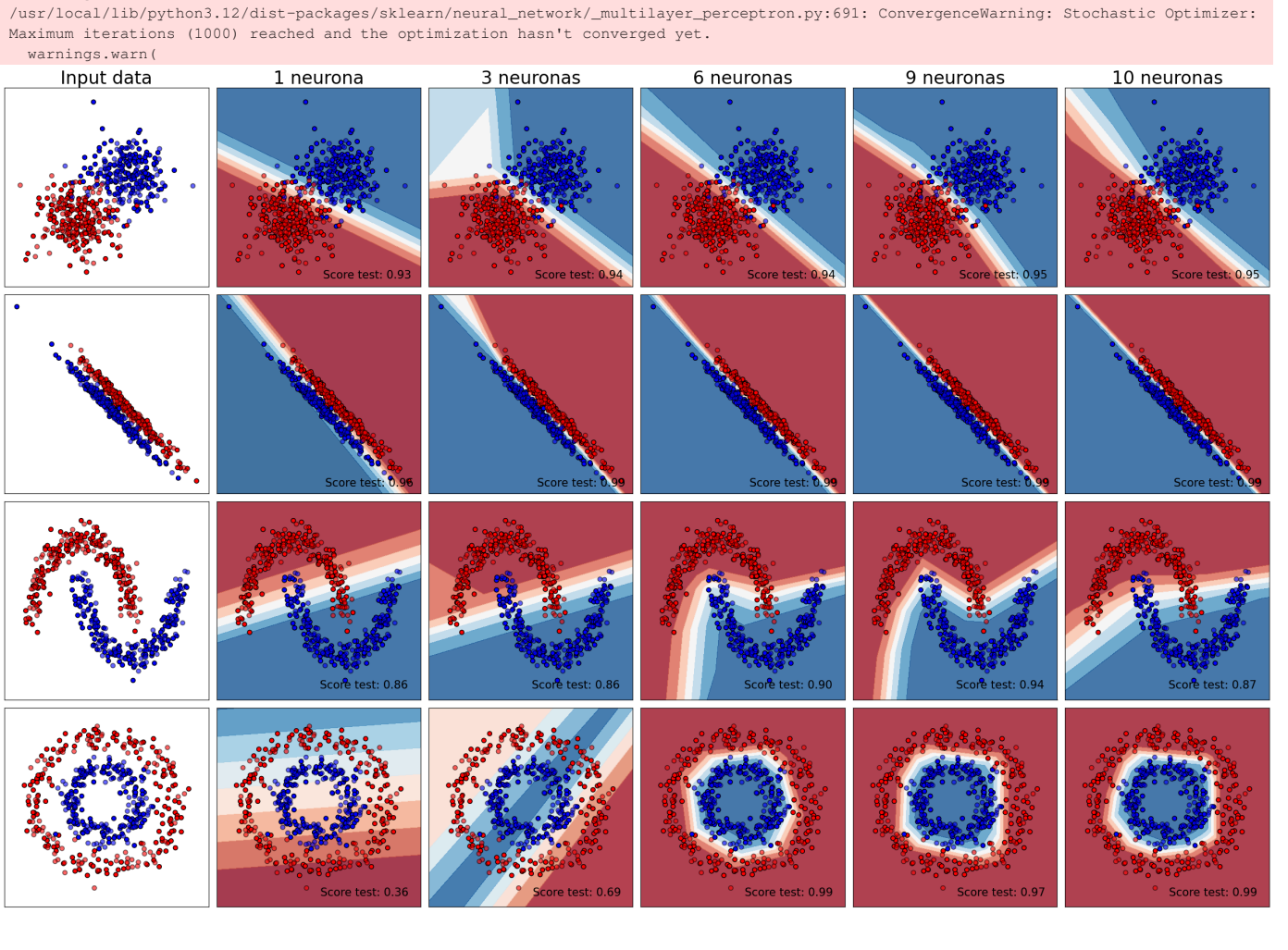
El modelo que más le cuesta ajustar es el de moons, por la propia forma de los datos, donde observamos un descenso del error significativo para 3 neuronas, donde se estabiliza en ese umbral, con un error mínimo en torno al 9%. Los conjuntos de datos que mejor ajusta son los de datos separables linealmente y circles, como es lógico, por la propia forma de los datos, logrando un error cercano a 0.

Salvo para moons, tenemos en general muy buenas predicciones, comparando con los demás clasificadores.

3. **Haz una gráfica similar a las obtenidas en este cuaderno con `plot_classifiers(classifiers, datasets)` para cinco de las distintas redes neuronales entrenadas. Elige las cinco que ilustren tus conclusiones.**

```
# 3. Gráfica para cinco clasificadores de redes neuronales
# con distinto número de neuronas.

# Seleccionamos clasificadores

k_selected = [
    ("1 neurona", clf_neural_networks[0]),
    ("3 neuronas", clf_neural_networks[2]),
    ("6 neuronas", clf_neural_networks[5]),
    ("9 neuronas", clf_neural_networks[8]),
    ("10 neuronas", clf_neural_networks[9])
]

plot_classifiers(k_selected, datasets)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
```



4. **Comenta la forma de la frontera de decisión dependiendo del número de neuronas en la capa oculta.**

Como habíamos comentado observando las gráficas del error, se puede ver claramente por qué el conjunto de moons es el que peor ajusta el modelo. Al tratar de trazar líneas para ajustar los datos, por la forma de este, le cuesta mucho captar la dispersión de los datos.

En general, con el aumento de neuronas, la frontera de decisión adopta formas más flexibles. Esto se puede observar muy bien para circles, donde la frontera en una neurona (regresión lineal) consiste en rectas paralelas entre sí, y a medida que aumentamos el número de estas, capta mejor la forma de los datos, estando más segura de su clasificación.

Para los dos primeros conjuntos de datos, al tratarse de una línea diagonal más o menos bien definida la que separa ambas clases, vemos que la red neuronal distingue muy bien para estos casos desde los modelos más simples posibles.

Ejercicio 1.4: Exploración de redes neuronales profundas

1. Haz una gráfica de la dependencia del error de clasificación de una red neuronal en función del número de capas ocultas `hidden_layer_sizes = ((5,), (5, 5,), (5, 5, 5,),...)`

2. Comenta la dependencia del error de clasificación con el número de capas ocultas.

3. Haz una gráfica similar a las obtenidas en este cuaderno con `plot_classifiers(classifiers, datasets)` para cinco de las distintas redes neuronales entrenadas. Elige las cinco que ilustren tus conclusiones.

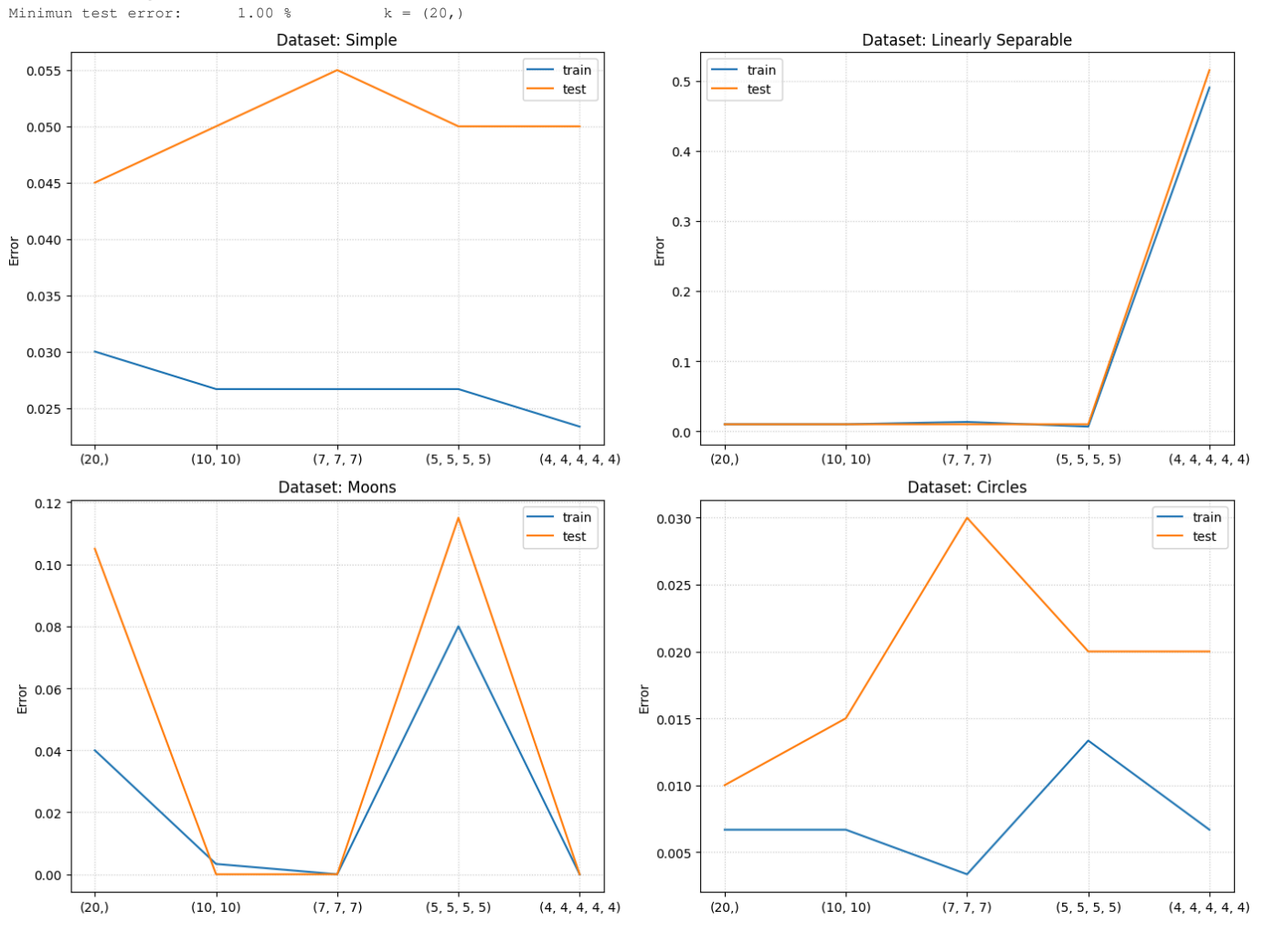
4. Comenta la forma de la frontera de decisión dependiendo del número de capas ocultas.

Resolución del Ejercicio 1.4

1. **Haz una gráfica de la dependencia del error de clasificación de una red neuronal en función del número de capas ocultas `hidden_layer_sizes = ((5,), (5, 5,), (5, 5, 5,),...)`**

```
# 1. Gráfica del clasificador de redes neuronales ocultas
# Repartimos el mismo número de neuronas (20) entre capas ocultas de forma diferente

hidden_layer_sizes = ((20,), (10, 10,), (7, 7, 7,), (5, 5, 5, 5,), (4, 4, 4, 4, 4,))

clf_neural_networks_deep = [
    Pipeline([
        ('scaler', StandardScaler()),
        ('mlp', MLPClassifier(
            hidden_layer_sizes=k,
            max_iter=1000,
            alpha=0.0,
            random_state=42
        ))
    ])

    for k in hidden_layer_sizes]

plot_error(classifiers=clf_neural_networks_deep,
           datasets=datasets,
           hyperparameters=[str(x) for x in hidden_layer_sizes],
           names=nombres)
```

```
--- Dataset: Simple ---
Minimun training error:  2.33 %          k = (4, 4, 4, 4, 4)
Minimun test error:      4.50 %          k = (20,)
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
--- Dataset: Linearly Separable ---
Minimun training error:  0.67 %          k = (5, 5, 5, 5)
Minimun test error:      1.00 %          k = (20,)
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
--- Dataset: Moons ---
Minimun training error:  0.00 %          k = (7, 7, 7)
Minimun test error:      0.00 %          k = (10, 10)
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
--- Dataset: Circles ---
Minimun training error:  0.33 %          k = (7, 7, 7)
Minimun test error:      1.00 %          k = (20,)
```



2. **Comenta la dependencia del error de clasificación con el número de capas ocultas.**

En este ejercicio hemos optado por tener siempre el mismo número de neuronas y repartirlas entre capas en diferentes formas, para ver cómo se comporta el error en test y en train. En un primer vistazo observamos diferencias significativas según el Dataset utilizado. En `Moons` y en `Linearly separable` vemos como ambos errores en test llegan a tocar el 0, es decir con 2 o 3 capas el modelo ya se encuentra en su punto óptimo y es donde mejor generaliza y resuelve los problemas para estos 2 datasets. Esto se debe a que modelos con más capas son capaces de adaptarse con mayor facilidad a los datos y representar funciones más complejas. En este caso las 2 capas permiten al modelo "doblar" el espacio y llegar a captar esta estructura no lineal que tienen las lunas.

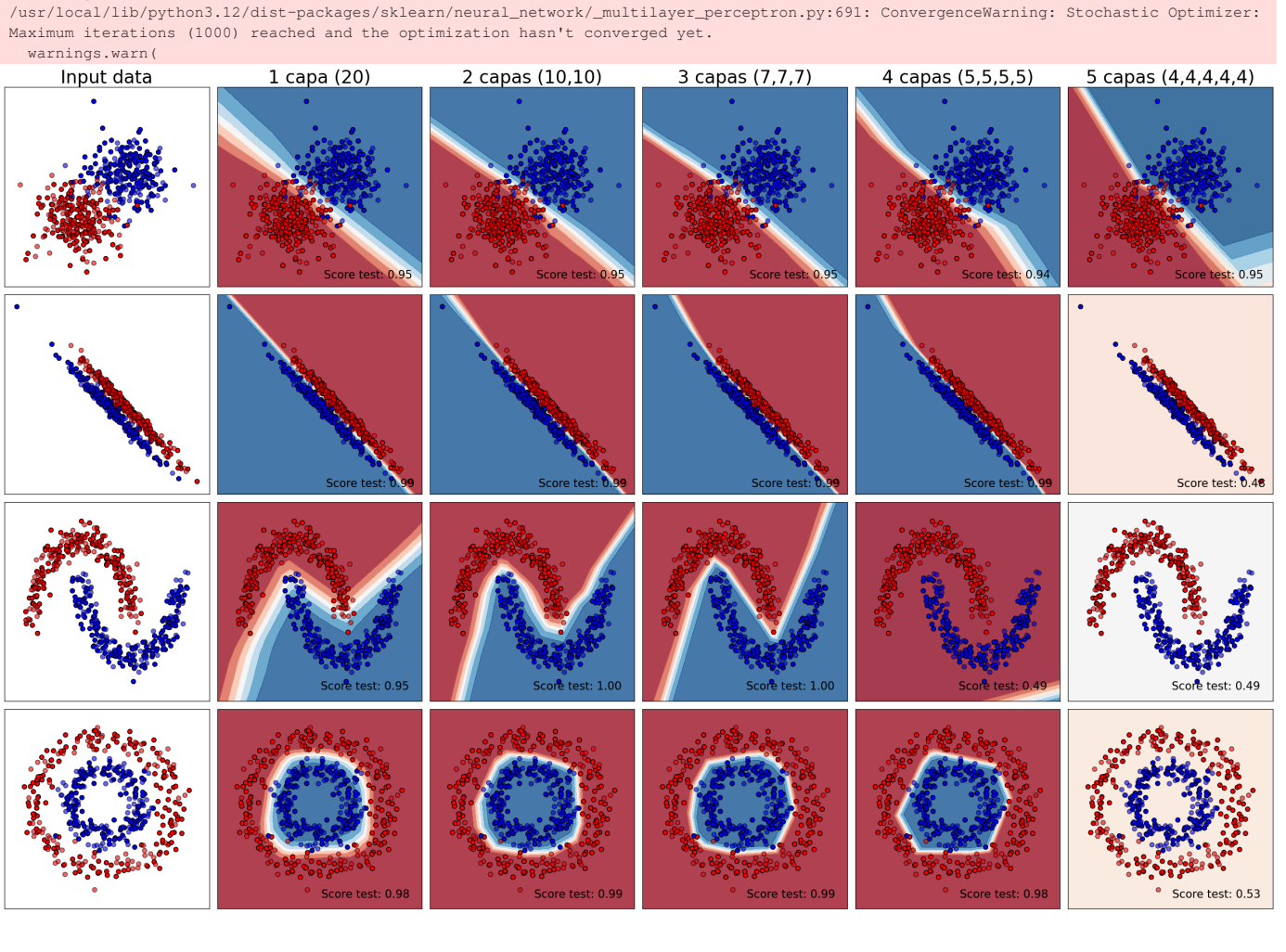
Sin embargo, vemos cómo en `Linearly separable` , a partir de cierto punto el error se dispara; este es realmente, una función muy sencilla e incluso con una función lineal y una sola capa era suficiente para tener un 0 de error, sin embargo, al aumentar la complejidad con 4 y 5 capas, el modelo deja de ver patrones generales y empieza a "ver fantasmas" donde solo hay ruido o "outliers" que el modelo está tratando de capturar en su función. Esto es lo que se conoce como sobreajuste `overfitting` ; al añadirle tantas capas neuronales se produce un sobreanálisis de los datos y resulta extremadamente contraproudcente.

Aunque en la mayoría de datasets el error tiende a crecer por la restricción de anchura, en `Moons` observamos una excepción fascinante: al llegar a la estructura (5,5,5,5) se alcanza un pico de error, pero sorprendentemente el modelo lo corrige y vuelve a generalizar mejor con (4,4,4,4,4). Este comportamiento recuerda al fenómeno del Double Descent (Belkin et al. (2019)) (o al menos, una dinámica no monótona de la generalización); teóricamente ese pico se asociaría al umbral de interpolación, aunque aquí probablemente refleje inestabilidad en la optimización ya que el fenómeno del Double Descent puro se produce cuando el error en train se mantiene en 0, y en nuestro caso también sucede un repunte en el mismo. Contrario a la teoría estadística clásica, el exceso de parámetros no degrada el modelo, sino que le otorga la libertad necesaria para encontrar la explicación más simple y menos errática compatible con las observaciones. Este proceso de selección de la función más suave reduce el error en test (y de generalización), demostrando que la sobre-parametrización en ocasiones puede ser beneficiosa para predecir mejor, que al final es el objetivo último del aprendizaje automatizado.

3. **Haz una gráfica similar a las obtenidas en este cuaderno con `plot_classifiers(classifiers, datasets)` para cinco de las distintas redes neuronales entrenadas. Elige las cinco que ilustren tus conclusiones.**

```
# 3. Gráfico de las fronteras de decisión

neuronas = [
    ("1 capa (20)", clf_neural_networks_deep[0]),
    ("2 capas (10,10)", clf_neural_networks_deep[1]),
    ("3 capas (7,7,7)", clf_neural_networks_deep[2]),
    ("4 capas (5,5,5,5)", clf_neural_networks_deep[3]),
    ("5 capas (4,4,4,4,4)", clf_neural_networks_deep[4])
]

plot_classifiers(neuronas, datasets)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
```



4. **Comenta la forma de la frontera de decisión dependiendo del número de capas ocultas.**

En general, con 20 neuronas al modelo le basta para captar la estructura de los datos en todos los casos, por eso siempre se obtienen puntuaciones muy altas en el `score` . Al mantener el número de neuronas iguales, el análisis se limita a comparar diferencia meramente estructurales, esto es, la forma en la que repartimos las neuronas entre capas. Vale la pena mencionar cómo en `circles` y en `moons` la frontera de decisión es suave y curva, sin embargo al añadir más capas ocultas, las fronteras se hacen más rígidas y con cortes más secos, esto es lo que antes era un círculo con 5 capas se convierte en una especie de heptágono. Esto se debe a que las redes neuronales emplean funciones de activación para aprender modelos no lineales. En otras palabras cada neurona en una capa oculta actúa como un clasificador lineal que traza una "recta" en el espacio de los datos. Al tener 20 rectas para rodear una zona (como la mancha azul central en el círculo), el resultado es un polígono de 20 lados que, visualmente, es tan suave que aproxima perfectamente un círculo. Cuando la red es profunda pero estrecha (pocas neuronas por capa), el proceso cambia: ahora el modelo tan solo dispone de 4 rectas para intentar hacer un círculo.

Otra observación al visualizar los gráficos, la cual invalida lo comentado en el ejercicio anterior, es que el modelo no falla porque "vea fantasmas", en nuestro caso el modelo de redes neuronales falla porque empeora la comprensión geométrica, es decir, no es un problema de **varianza** es un problema de **sesgo**. Para "ver fantasmas" donde solo hay ruido, el modelo necesita memorizar estos outliers, y la memoria es la misma en todos los casos porque la memoria depende del número de neuronas (amplitud del modelo).

Otro caso destacable es el modelo con 5 capas en el problema linearmente separable, lunas y círculos, el modelo más complejo lo hace muy mal en estos 3 casos. Esto se debe a la función de activación. Al no especificar el hiperparámetro de la función de activation dentro de `MLPClassifier` , el modelo utiliza su valor por defecto, que en la librería `scikit-learn` es `relu` . La función ReLU (Rectified Linear Unit) transforma los valores de las neuronas devolviendo el máximo entre cero y el valor de entrada, es decir, $f(x) = max(0, x)$. Entonces al ser negativo, la función devuelve un 0, y cuando las 4 neuronas de una capa sacan un 0, el modelo "colapsa" y deja de aprender y los pesos w nunca se actualizan.